

Data Science Capstone Project Baseline Model

Course: MSDS640 Data Science Capstone · Dataset: UCI Adult Census Income (1994 U.S. Census) · Erik Herb

This is the baseline modeling milestone, where I train the simplest reasonable model as an honest benchmark that every later model has to beat. The model is logistic regression, which is my interpretable baseline and the null case for Hypothesis 3. The task is to predict whether a person earns over 50K a year from their demographic, education, and job information, framed as a lender that flags likely high earners. Because of that framing I care about which features drive the decision and how many of the real high earners I catch, not raw accuracy. As in the earlier milestones, the data is loaded into SQLite and pulled back out before any modeling.

The project tests three hypotheses, and this milestone sits under the third. Hypothesis 1 is that education level drives the over 50K prediction more than hours worked. Hypothesis 2 is that XGBoost is the fairest of the three models across nativity, with the smallest US-born versus foreign-born recall gap on true high earners. Hypothesis 3 is that XGBoost beats both the logistic baseline and the MLP on held-out PR-AUC and F1. This milestone builds the logistic regression baseline that Hypothesis 3 is measured against.

1. Loading and preparing the data

I pull the data back out of the SQLite database and rebuild the cleaned, feature-engineered table from my EDA milestone rather than re-arguing any of it here. That is the same preprocessing I justified in the last report: an Unknown bucket for the missing categoricals, dropping the 24 exact duplicates while the sampling weight is still in the table, then dropping the sampling weight and the redundant text education column, cleaning the target label, and adding the native region and capital flags. The load pulls 32,561 rows, and after dropping the duplicates there are 32,537 rows across 13 columns. The positive rate is 0.241, so about 24 percent of people are high earners, which is the imbalance the rest of the milestone is built around.

I one-hot encode the eight categorical columns and join them to the seven numeric columns, which gives a model matrix of 52 columns. That matrix and the 0/1 target are what every model below trains on.

2. The model I chose and why

Logistic regression is the right baseline here for a few reasons. The data is about 52 mostly binary columns plus a few numeric ones, dense and tabular, with no images, text, or time series, so nothing here calls for a fancier model. The task is binary classification, and logistic regression is built for exactly that. My lender use case has to be able to explain what the model keyed on, and logistic regression trains in seconds and gives one signed coefficient per feature, so it is a strong baseline choice.

It works by taking a weighted sum of the features, passing that through the sigmoid to get a probability, and applying a 0.5 threshold for a yes or no. The one limit that matters is that it is linear and additive, so it misses interactions and curves unless I hand-engineer them. That limit is the whole reason the later milestones try more flexible models, and it is what Hypothesis 3 is testing.

3. The split, and how I handle the class imbalance

I use a three-way stratified split instead of a single train and test split, so I have a dedicated validation set to make decisions on before I ever touch the test set. The training set is 60 percent, the validation set is 20 percent, and the test set is 20 percent, and I stratify all three so the 0.241 positive rate is preserved everywhere. That gives 19,521 training rows, 6,508 validation rows, and 6,508 test rows. The training set is what the models fit on, the validation set is where I compare the raw and SMOTE versions and tune the hyperparameters, and the test set stays untouched until the final

evaluation so the last number is honest.

For the imbalance I oversample the minority class with SMOTE on the training rows rather than undersampling the majority, because undersampling would throw away most of the real under 50K rows while oversampling keeps every real row. The rule that matters is that resampling happens after the split and only on training rows. I put the scaler, SMOTE, and the model into a single pipeline so that SMOTE is refit only on the training data and never sees a validation or test row, which is what keeps the validation and test numbers honest.

4. The metrics I use to evaluate

I define the metrics once here, since every result below reports the same ones. Writing TP, TN, FP, and FN for true and false positives and negatives:

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ Precision = $TP / (TP + FP)$ Recall = $TP / (TP + FN)$ F1 = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Accuracy is the overall share correct, which is misleading here because always guessing the majority class already scores about 76 percent. Precision is the share of the people I flag as high earners who really are high earners. Recall is the share of the real high earners I actually catch. F1 is the harmonic mean of precision and recall, so it stays low unless both are decent. PR-AUC, the area under the precision-recall curve, is my lead metric because it focuses on the positive class directly and has a no-skill floor at the 0.241 positive rate. ROC-AUC is the area under the true positive versus false positive rate curve, which ranks across all thresholds but flatters on imbalanced data, so I keep it as a secondary number.

5. A naive baseline to beat

Before the real model I fit the most naive predictor there is, one that always guesses the majority class, and I check it on the validation set. This is the floor I have to clear to show the model is learning anything, and it makes the accuracy trap concrete. It scores 0.759 accuracy while catching zero real high earners, and its PR-AUC just sits at the 0.241 base rate, so there is no signal at all. A useless model already scores in the high 70s on accuracy, and the recall of zero is what gives it away. Everything below has to clear this floor.

6. The untuned baseline: raw imbalance versus SMOTE

Next I fit the actual model with default settings as my before-state, and I run it two ways on the same validation rows: once on the raw imbalance, then with SMOTE added, so the effect of handling the imbalance is visible on its own.

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Raw imbalance	0.853	0.737	0.608	0.666	0.769	0.909
With SMOTE	0.815	0.580	0.846	0.688	0.767	0.908

On the raw imbalance the model catches only about 61 percent of the real high earners. Adding SMOTE lifts recall into the mid 80s, so I catch far more of them. The cost is some precision, which means more false alarms, and F1 ticks up only slightly. PR-AUC and ROC-AUC barely move, because SMOTE shifts where the 0.5 cutoff lands through the coefficients it relearns rather than changing how well the model ranks people. For a lender that would rather flag a borderline case than miss a real high earner, that is the right trade, so I carry SMOTE forward.

7. Cross-validating the baseline

A single validation number can be lucky, so alongside the held-out validation set I run a 5-fold stratified cross-validation on the training data as a stability check. Five folds keep enough minority-class rows per fold for a steady PR-AUC, and stratifying holds the 24 percent positive rate in every fold. Passing the whole pipeline in refits SMOTE inside each fold, so there is no leakage. This is a supplementary check on the training data, and the validation set stays my main tool for comparing models and tuning.

Metric	Mean	Std	Variance
PR-AUC	0.773	0.011	0.00011
F1	0.681	0.011	0.00013
Recall	0.837	0.015	0.00024

Cross-validated PR-AUC sits around 0.77 with a standard deviation of about 0.01, and F1 and recall are just as tight across folds. A spread that small means the baseline is stable rather than riding on one lucky split, so the validation numbers hold up as a benchmark.

8. Tuning the hyperparameters with grid search

Logistic regression has few knobs, so I tune the two that shape it. I fit each combination on the training set and score it on the validation set with PR-AUC, then keep the best. C is the inverse regularization strength: a small C means stronger regularization and a simpler model that risks underfitting, while a large C means weaker regularization that fits the training data more closely and risks overfitting. The penalty is the other knob, where L2 shrinks all coefficients smoothly and L1 can zero some out and act like a light feature selector, so it is worth trying against L2. I search with the liblinear solver.

The search lands on $C = 10$ with an L1 penalty and a best validation PR-AUC of 0.768. To see the underfitting-to-overfitting picture directly I also plot a validation curve over C , with the model scored on the training rows and on the validation set. Both lines sit at about 0.77 to 0.78 across every value of C with only a small gap between them, so the model is not overfitting. It separates the classes about as well on unseen people as on training people. Comparing tuned against untuned on the same validation rows:

Model	Accuracy	Precision	Recall	F1	PR-AUC	ROC-AUC
Untuned (default C , L2)	0.815	0.580	0.846	0.688	0.767	0.908
Tuned ($C = 10$, L1)	0.816	0.581	0.847	0.689	0.768	0.908

Tuning C and the penalty moves PR-AUC and F1 by only a thousandth or two, and the validation curve is flat by about $C = 1$. That is a bit of a letdown, but it is informative: logistic regression is low variance and already near its ceiling on this data. A real jump has to come from a more flexible model, which is what Hypothesis 3 tests, not from squeezing this one harder.

9. Evaluating the final baseline

I settle on the tuned pipeline and evaluate it on the held-out test set, which I touch here for the first time, using the metrics defined earlier, with PR-AUC as the lead number and recall and F1 alongside it. On the over 50K class it reaches 0.58 precision and 0.87 recall for an F1 of 0.70, with a PR-AUC of 0.795 and a ROC-AUC of 0.916. These come in a little above the validation and cross-validation estimates, which sat around 0.77, because the particular test split lands slightly easier, but they are close enough to confirm the baseline is stable. The confusion matrix shows the baseline catching most of the real high earners while taking a moderate number of false alarms, which is the

recall-first trade I made on purpose. The precision-recall curve stays well above the 0.241 no-skill line the whole way, so the baseline has real signal on the minority class, and the ROC curve bows far from the random diagonal, so it separates the two classes well overall.

Here is the final baseline set against the naive floor, all on the test set:

Model	Recall	F1	PR-AUC	ROC-AUC
Naive (majority class)	0.000	0.000	0.241	0.500
Raw imbalance LR	0.628	0.681	0.795	0.916
Final baseline (tuned)	0.867	0.697	0.795	0.916

The naive model catches zero high earners, and my baseline clears it by a wide margin on recall, F1, and PR-AUC. This is the benchmark that XGBoost and the MLP now have to beat.

10. Learning curve: is more data the answer

The validation curve already ruled out overfitting, so the learning curve asks the other question, which is whether more rows would help. I plot PR-AUC against training set size, scoring each fit on its own training subset and on the validation set. The validation curve climbs fast on the first few thousand rows and then flattens, with the training curve only slightly above it. A small final gap plus a flat end means the model is not starved of data, so more rows would not buy much. The baseline is limited by the model and the features, not the amount of data, which again points to a more flexible model as the next step rather than more rows.

11. A quick error analysis

Beyond the summary numbers I want to see who the model gets wrong, so I split the misclassified test rows by error type and compare their real feature values against the high earners the model caught. There are 209 missed high earners, 1,359 caught high earners, and 973 false alarms. The table shows the average of each group on the model's strongest features:

Feature	Missed >50K (FN)	Caught >50K (TP)	False alarm (FP)
education_num	9.8	11.9	10.8
capital_gain	206.9	4921.3	238.9
hours_per_week	41.5	45.9	45.4
age	41.2	44.8	43.3

The high earners the model misses look ordinary on its strongest features. They have lower education, fewer hours, are younger, and have almost no capital gain, with about 95 percent of the missed ones sitting at zero capital gain against 75 percent of the ones it caught. So capital gain does a lot of the work, and the model stumbles on high earners who got there without it. The false alarms are the mirror image, people who look like high earners on paper but are not. That is exactly what a linear model reading each feature on its own would do, and it points to where a feature-combining model can help.

12. Training and prediction time

I want the baseline's cost on record before comparing it against heavier models later, so I time the final pipeline. It fits in about 2.8 seconds and predicts in about 0.006 seconds on the 6,508 test rows. That matters for the comparison

ahead: if XGBoost or the MLP only edge it out on PR-AUC and F1 but cost far more to train and are harder to explain, the small gain has to be weighed against that. That trade-off is part of what Hypothesis 3 is really about.

13. Where the baseline falls short, and what comes next

The main flaw is built in. Logistic regression is linear and additive, so it misses the curves and interactions my EDA already found, like the rise and fall of the over 50K rate with age and the way education and marital status matter together rather than separately. It also leans hard on a few strong features, capital gain most of all, and misses high earners who lack them. XGBoost finds non-linear splits and interactions on its own, so I expect it to win on PR-AUC and F1, which is Hypothesis 3, and the trade-off it makes is giving up the transparency and fast training that keep logistic regression as the baseline. The next steps are threshold tuning, training XGBoost and the MLP with significance tests behind the comparison, and then the nativity fairness audit for Hypothesis 2.

14. Reproducibility

One fixed seed on the split and the models, and a single load path from the database, mean the notebook reruns to the same numbers. I ran on random seed 42 with numpy 2.4.4, pandas 3.0.2, scikit-learn 1.8.0, imbalanced-learn 0.14.2, and scipy 1.17.1.

How I used an LLM for this assignment

I used an LLM to help debug my code and troubleshoot errors, and to format the output of my code so it showed the results the way I wanted. It also helped me create some of the visual plots in the notebook, and I used it to help me correctly interpret the results. I wrote and edited the analysis myself and confirmed the numbers by running the notebook top to bottom.